

CS 311: Algorithm Design and Analysis — Lecture 3

Insertion Sort, Divide and Conquer, Binary Search, Mergesort, Master Theorem — Study Questions

Batch 1 – Questions 1 to 16: Insertion Sort

Q1. Which topics were covered in the previous lecture, as recalled at the start?

- a) Compilers, parsing, and code generation
- b) Binary trees, heaps, and tries
- c) Graph algorithms, dynamic programming, and hashing
- d) Analysis of algorithms, asymptotic notation, and insertion sort

Answer: d) Analysis of algorithms, asymptotic notation, and insertion sort

Q2. Insertion sort is described as what kind of algorithm?

- a) A divide-and-conquer algorithm
- b) A greedy algorithm
- c) An incremental algorithm
- d) A randomized algorithm

Answer: c) An incremental algorithm

Q3. In the slides' pseudocode, the outer loop of InsertionSort starts at:

- a) $i = 2$
- b) $i = 1$
- c) $i = 0$
- d) $i = n$

Answer: a) $i = 2$

Q4. What does each iteration of the outer loop store in 'key'?

- a) The value $A[n]$
- b) The index i
- c) The value $A[i]$
- d) The value $A[1]$

Answer: c) The value $A[i]$

Q5. In the pseudocode, j is initially set to:

- a) 0
- b) $i + 1$
- c) n
- d) $i - 1$

Answer: d) $i - 1$

Q6. What is the while-loop condition in InsertionSort?

- a) $j > 0$ and $A[j] > \text{key}$
- b) $j > 0$ only
- c) $j < n$ and $A[j] < \text{key}$
- d) $A[j] = \text{key}$

Answer: a) $j > 0$ and $A[j] > \text{key}$

Q7. What does the body of the while loop do?

- a) Increments i
- b) Swaps $A[j]$ with $A[n]$

c) Deletes $A[j]$

d) Shifts $A[j]$ one position to the right ($A[j+1] = A[j]$) and decrements j

Answer: d) Shifts $A[j]$ one position to the right ($A[j+1] = A[j]$) and decrements j

Q8. After the while loop ends, what is done with key?

a) $A[j] = \text{key}$

b) $A[j+1] = \text{key}$

c) key is discarded

d) $A[1] = \text{key}$

Answer: b) $A[j+1] = \text{key}$

Q9. What is the loop invariant stated in the lecture?

a) $A[1..i]$ is reversed

b) $A[i..n]$ is sorted and $A[1..i-1]$ is untouched

c) $A[1..n]$ is always sorted

d) $A[1..i-1]$ is sorted and $A[i..n]$ is untouched

Answer: d) $A[1..i-1]$ is sorted and $A[i..n]$ is untouched

Q10. In the first picture example (11 4 5 2 15 7), what is the array after the first step?

a) 2 4 5 11 15 7

b) 4 11 5 2 15 7

c) 4 5 11 2 15 7

d) 11 4 5 2 7 15

Answer: b) 4 11 5 2 15 7

Q11. Which is the final sorted array in that example?

a) 2 4 5 7 11 15

b) 4 2 5 7 11 15

c) 2 4 5 11 15 7

d) 15 11 7 5 4 2

Answer: a) 2 4 5 7 11 15

Q12. In the trace of 30 10 40 20, what are i , j , and key at the first outer iteration ($i = 2$)?

a) $i = 2$, $j = 2$, key = 40

b) $i = 2$, $j = 1$, key = 10

c) $i = 1$, $j = 0$, key = 30

d) $i = 3$, $j = 1$, key = 20

Answer: b) $i = 2$, $j = 1$, key = 10

Q13. What is the final array in the trace of 30 10 40 20?

a) 10 30 40 20

b) 10 20 30 40

c) 40 30 20 10

d) 20 10 30 40

Answer: b) 10 20 30 40

Q14. In the worst case for insertion sort, how many while-loop iterations does the i -th step take (t_i)?

a) 1 iteration

b) n iterations always

c) 0 iterations

d) i iterations (the input is reverse sorted)

Answer: d) i iterations (the input is reverse sorted)

Q15. What does the sum of i for $i = 2$ to n equal?

- a) $2n$
- b) n
- c) $n \log n$
- d) $n(n+1)/2 - 1$, which is $\Theta(n^2)$

Answer: d) $n(n+1)/2 - 1$, which is $\Theta(n^2)$

Q16. What is the worst-case running time of insertion sort?

- a) $\Theta(n \log n)$
- b) $\Theta(n)$
- c) $\Theta(\log n)$
- d) $\Theta(n^2)$

Answer: d) $\Theta(n^2)$

Batch 2 - Questions 17 to 31: Divide and Conquer and Binary Search

Q17. What are the three steps of the divide-and-conquer paradigm?

- a) Initialize, iterate, and terminate
- b) Read, write, and execute
- c) Sort, search, and merge
- d) Divide, conquer (solve subproblems recursively), and combine

Answer: d) Divide, conquer (solve subproblems recursively), and combine

Q18. In the divide step, we:

- a) Combine the solutions
- b) Divide the problem into subproblems
- c) Sort the output
- d) Delete the input

Answer: b) Divide the problem into subproblems

Q19. In the conquer step, we:

- a) Merge the solutions
- b) Solve the subproblems recursively
- c) Ignore the subproblems
- d) Divide the input in half once

Answer: b) Solve the subproblems recursively

Q20. In the divide-and-conquer technique picture, a problem of size n is split into:

- a) n subproblems of size 1
- b) Two subproblems of size $n/2$
- c) One subproblem of size $n-1$
- d) Three subproblems of size $n/3$

Answer: b) Two subproblems of size $n/2$

Q21. Which of the following is listed as a divide-and-conquer example?

- a) Mergesort and quicksort
- b) Selection sort and counting sort
- c) Insertion sort and bubble sort
- d) Heap sort and radix sort

Answer: a) Mergesort and quicksort

Q22. Which of the following is also listed as a divide-and-conquer example?

- a) Floyd-Warshall algorithm
- b) Strassen's algorithm for matrix multiplication
- c) Dijkstra's algorithm
- d) Kruskal's algorithm

Answer: b) Strassen's algorithm for matrix multiplication

Q23. Which other problems are listed as divide-and-conquer examples?

- a) Binary search, powering a number, and the closest pair problem
- b) Knapsack, coin change, and edit distance
- c) Huffman coding and the activity selector
- d) Topological sort and Euler tours

Answer: a) Binary search, powering a number, and the closest pair problem

Q24. What does binary search find?

- a) The median of an unsorted array
- b) An element in a sorted array
- c) A cycle in a graph
- d) The maximum of any array

Answer: b) An element in a sorted array

Q25. In binary search, what is the 'divide' step?

- a) Check the middle element
- b) Sort the array
- c) Merge two halves
- d) Pick a random element

Answer: a) Check the middle element

Q26. In binary search, what is the 'conquer' step?

- a) Count the elements
- b) Recursively search one sub-array
- c) Sort the sub-arrays
- d) Search both sub-arrays

Answer: b) Recursively search one sub-array

Q27. In binary search, what is the 'combine' step?

- a) Merging both halves
- b) Sorting the result
- c) Linear in the array size
- d) Trivial

Answer: d) Trivial

Q28. In the binarySearch code, mid is computed as:

- a) $(low + high) / 2$
- b) $(low - high) / 2$
- c) $size / 3$
- d) $low \times high$

Answer: a) $(low + high) / 2$

Q29. In the code, if $a[mid] < x$, what happens?

- a) $low = mid + 1$
- b) return mid

- c) return -1
- d) high = mid - 1

Answer: a) low = mid + 1

Q30. In the 'Find 9' example on the array 3 5 7 8 9 12 15, which element does the given code examine first?

- a) 9
- b) 12
- c) 8 (mid = $(0 + 6)/2 = 3$)
- d) 3

Answer: c) 8 (mid = $(0 + 6)/2 = 3$)

Q31. What does the binarySearch code return if the element is not found?

- a) 0
- b) mid
- c) size
- d) -1

Answer: d) -1

Batch 3 - Questions 32 to 51: Mergesort and the Master Theorem

Q32. What is the basic strategy of mergesort?

- a) Insert each element into a sorted prefix
- b) Pick a pivot and partition
- c) Repeatedly select the minimum
- d) Sort two sub-arrays recursively and merge the sorted arrays into one

Answer: d) Sort two sub-arrays recursively and merge the sorted arrays into one

Q33. In the merge step, we repeatedly:

- a) Compare the first elements of the remaining unprocessed portions and copy the smaller into A
- b) Sort each array again
- c) Copy the larger element first
- d) Swap adjacent elements

Answer: a) Compare the first elements of the remaining unprocessed portions and copy the smaller into A

Q34. When one of the arrays has been completely processed in merging, we:

- a) Sort the remaining elements
- b) Stop and discard the rest
- c) Copy the remaining unprocessed elements from the other array into A
- d) Start over

Answer: c) Copy the remaining unprocessed elements from the other array into A

Q35. In the mergeSort pseudocode, mid is computed as:

- a) $(\text{first} - \text{last}) / 2$
- b) $\text{first} \times \text{last}$
- c) $(\text{first} + \text{last}) / 2$
- d) $\text{last} - \text{first}$

Answer: c) $(\text{first} + \text{last}) / 2$

Q36. In the mergeSort pseudocode, the recursive calls are on:

- a) E[first..last] twice
- b) E[first..first] only
- c) E[mid..mid] only
- d) E[first..mid] and E[mid+1..last], followed by merge

Answer: d) E[first..mid] and E[mid+1..last], followed by merge

Q37. When does the mergeSort pseudocode do any work (recursion)?

- a) When first < last
- b) When first > last
- c) Always
- d) When first = last

Answer: a) When first < last

Q38. What recurrence expresses the cost of merge sort in the slides?

- a) $T(n) = 2T(n-1) + \Theta(n)$
- b) $T(n) = T(n/2) + \Theta(1)$
- c) $T(n) = T(n-1) + \Theta(1)$
- d) $T(n) = 2T(n/2) + \Theta(n)$ for $n > 1$, with $T(1) = 0$

Answer: d) $T(n) = 2T(n/2) + \Theta(n)$ for $n > 1$, with $T(1) = 0$

Q39. What does that recurrence solve to?

- a) $\Theta(\lg n)$
- b) $\Theta(n^2)$
- c) $\Theta(n)$
- d) $\Theta(n \lg n)$

Answer: d) $\Theta(n \lg n)$

Q40. In merge sort, the combine step is:

- a) Trivial
- b) Logarithmic
- c) Quadratic
- d) A linear-time merge

Answer: d) A linear-time merge

Q41. In merge sort, the divide step is:

- a) Quadratic
- b) Linear
- c) Trivial
- d) Logarithmic

Answer: c) Trivial

Q42. The efficiency of mergesort in all cases is:

- a) $\Theta(\log n)$
- b) $\Theta(n \log n)$
- c) $\Theta(n^2)$ in the worst case only
- d) $\Theta(n)$ in the best case

Answer: b) $\Theta(n \log n)$

Q43. What is the space requirement of mergesort?

- a) $\Theta(n)$, and it is not in-place
- b) $\Theta(\log n)$, in-place
- c) $\Theta(1)$, in-place

d) $\Theta(n^2)$

Answer: a) $\Theta(n)$, and it is not in-place

Q44. How many comparisons does mergesort use compared with the theoretical minimum for comparison-based sorting?

- a) The number is close to the theoretical minimum, about $n \lg n - 1.44n$
- b) n^2 comparisons
- c) Far above the theoretical minimum
- d) Exactly n comparisons

Answer: a) The number is close to the theoretical minimum, about $n \lg n - 1.44n$

Q45. Mergesort can also be implemented:

- a) Without recursion (bottom-up)
- b) Only in-place
- c) Only recursively
- d) Only with a stack

Answer: a) Without recursion (bottom-up)

Q46. The master theorem applies to recurrences of the form:

- a) $T(n) = aT(n) + n$
- b) $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for constants $a > 0$, $b > 1$, $d \geq 0$
- c) $T(n) = T(n-1) + n$
- d) $T(n) = aT(n/b) + 2^n$

Answer: b) $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for constants $a > 0$, $b > 1$, $d \geq 0$

Q47. According to the master theorem, if $a < b^d$ then:

- a) $T(n) \in \Theta(n^d)$
- b) $T(n) \in \Theta(n^d \lg n)$
- c) $T(n) \in \Theta(n^{\log_b a})$
- d) $T(n) \in \Theta(1)$

Answer: a) $T(n) \in \Theta(n^d)$

Q48. According to the master theorem, if $a = b^d$ then:

- a) $T(n) \in \Theta(n^{\log_b a})$
- b) $T(n) \in \Theta(n^d \lg n)$
- c) $T(n) \in \Theta(\lg n)$
- d) $T(n) \in \Theta(n^d)$

Answer: b) $T(n) \in \Theta(n^d \lg n)$

Q49. According to the master theorem, if $a > b^d$ then:

- a) $T(n) \in \Theta(n)$
- b) $T(n) \in \Theta(n^d)$
- c) $T(n) \in \Theta(n^d \lg n)$
- d) $T(n) \in \Theta(n^{\log_b a})$

Answer: d) $T(n) \in \Theta(n^{\log_b a})$

Q50. Applying the master theorem to mergesort, $T(n) = 2T(n/2) + \Theta(n)$, we have $a = 2$, $b = 2$, $d = 1$. Which case applies and what is the result?

- a) $a < b^d$, giving $\Theta(n)$
- b) $a > b^d$, giving $\Theta(n^2)$
- c) No case applies
- d) $a = b^d$, giving $\Theta(n \lg n)$

Answer: d) $a = b^d$, giving $\Theta(n \lg n)$

Q51. For $T(n) = 4T(n/2) + \Theta(n)$ ($a = 4$, $b = 2$, $d = 1$), the master theorem gives:

- a) $\Theta(n)$
- b) $\Theta(n^3)$
- c) $\Theta(n \lg n)$
- d) $\Theta(n^2)$, since $a > b^d$ and $\log_2 4 = 2$

Answer: d) $\Theta(n^2)$, since $a > b^d$ and $\log_2 4 = 2$

Essay Questions

Q52. Explain insertion sort: how the algorithm works and why its worst-case running time is $\Theta(n^2)$.

Answer:

Insertion sort is an incremental algorithm. For $i = 2$ to n , it stores $\text{key} = A[i]$, sets $j = i - 1$, and, while $j > 0$ and $A[j] > \text{key}$, shifts $A[j]$ to $A[j+1]$ and decrements j ; finally it places key at $A[j+1]$. The loop invariant: $A[1..i-1]$ is sorted and $A[i..n]$ is untouched.

Worst case: the input is reverse sorted, so the while loop runs $t_i = i$ times for each i . Therefore $T(n) = \Theta(n + \sum t_i) = \Theta(n + \sum i) = \Theta(n + n^2) = \Theta(n^2)$, since the sum of i for $i = 2$ to n equals $n(n+1)/2 - 1$.

Q53. Describe the divide-and-conquer paradigm, and show how binary search fits into it.

Answer:

Divide-and-conquer has three steps: divide the problem into subproblems; conquer the subproblems by solving them recursively; combine the subproblem solutions. Examples: mergesort, quicksort, Strassen's matrix multiplication, binary search, powering a number, closest pair.

Binary search finds an element in a sorted array. Divide: check the middle element. Conquer: recursively search one sub-array (the left or the right, depending on the comparison). Combine: trivial. The code uses low , high , and $\text{mid} = (\text{low} + \text{high})/2$, and returns -1 if the element is not found.

Q54. Describe mergesort, give its recurrence and efficiency, and state its space requirement.

Answer:

Strategy: sort the two halves recursively and merge the two sorted halves. Merging repeatedly compares the first elements of the unprocessed portions and copies the smaller into the output; when one array is exhausted, the rest of the other is copied.

Divide: trivial ($\text{mid} = (\text{first} + \text{last})/2$). Conquer: recursively sort both halves. Combine: linear-time merge. Recurrence: $T(n) = 2T(n/2) + \Theta(n)$, $T(1) = 0$, which gives $\Theta(n \lg n)$ in all cases.

Number of comparisons is close to the theoretical minimum for comparison-based sorting ($\approx n \lg n - 1.44n$). Space: $\Theta(n)$, not in-place; it can be implemented without recursion (bottom-up).

Q55. State the master theorem and use it to solve $T(n) = 2T(n/2) + \Theta(n)$.

Answer:

If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for constants $a > 0$, $b > 1$, $d \geq 0$, then: (1) if $a < b^d$, $T(n) \in \Theta(n^d)$; (2) if $a = b^d$, $T(n) \in \Theta(n^d \lg n)$; (3) if $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$.

For mergesort: $a = 2$, $b = 2$, $d = 1$, so $b^d = 2 = a$ (case 2). Hence $T(n) \in \Theta(n^1 \lg n) = \Theta(n \lg n)$.